

AI Trip Planner - Memory Architecture Specification v1

1. Purpose

The AI Trip Planner shall maintain long-term memory about users, their travel preferences, travel history, and learned behaviors to provide increasingly personalized trip planning.

The memory system shall allow the planner to:

- Remember traveler profile information
- Remember past trips
- Learn user preferences
- Use historical travel experiences during itinerary generation
- Continuously improve recommendations over time

The memory system shall support retrieval of relevant context before planning and shall persist information across sessions.

2. Design Principles

1. Do not store all memory in a single document.
 2. Separate structured profile data from learned preferences.
 3. Store each trip as an independent memory object.
 4. Use structured queries whenever possible.
 5. Use vector search only for large unstructured content.
 6. Memory must be updateable independently.
 7. Planner agents must consume memory through a Memory Service API rather than querying storage directly.
-

3. Memory Types

3.1 Profile Memory

Represents stable user information.

Examples:

- Traveler names
- Family composition

- Dietary preferences
- Home location
- Hotel preferences
- Airline preferences
- Loyalty memberships

Example

```
{ "userId": "123", "travellers": [ { "name": "Munish", "relationship": "Self" }, { "name": "Megha", "relationship": "Spouse" }, { "name": "Amay", "relationship": "Child" } ], "dietPreference": "Vegetarian", "hotelCategory": "4-5 Star", "tripStyle": "Comfortable" }
```

3.2 Preference Memory

Represents learned or explicit preferences.

Examples:

- Prefers metro connectivity
- Avoids overnight buses
- Prefers premium hotels
- Likes South Mumbai
- Avoids excessive walking

Example

```
{ "memoryId": "pref001", "userId": "123", "memoryType": "Preference", "fact": "Avoid overnight buses", "source": "Conversation", "confidence": 0.95, "createdDate": "2026-06-20" }
```

3.3 Trip Memory

Represents completed or planned trips.

Each trip is stored independently.

Example

```
{ "tripId": "mumbai2026", "userId": "123", "destination": "Mumbai", "startDate": "2026-06-25", "endDate": "2026-06-28", "hotels": [], "activities": [], "userRating": 8 }
```

3.4 Trip Feedback Memory

Captures user feedback.

Examples:

- Hotel was excellent
- Itinerary was too rushed
- Walking distance too high

Example

```
{ "feedbackId": "f001", "tripId": "mumbai2026", "feedback": "Too much travel between attractions", "rating": 7 }
```

3.5 Conversation Memory

Stores recent conversations.

Purpose:

- Short-term continuity
- Preference extraction

Retention:

- Last N conversations
 - Summarized periodically
-

4. Storage Architecture

Preferred Storage:

Cosmos DB

Containers:

1. UserProfiles
2. Preferences
3. Trips
4. TripFeedback
5. ConversationSummaries

Optional Future Containers:

1. Embeddings
2. TravelDocuments
3. ReviewsCache

5. Memory Service

Planner agents must never access Cosmos DB directly.

Create a dedicated Memory Service.

Read Operations

GetUserProfile(userId)

GetPreferences(userId)

GetTripHistory(userId)

GetTripFeedback(userId)

GetTravelContext(userId)

Write Operations

SaveUserProfile()

SaveTrip()

SavePreference()

SaveFeedback()

SaveConversationSummary()

6. Travel Context API

Primary API used by itinerary agents.

Function:

GetTravelContext(userId)

Returns:

```
{ "profile": {}, "preferences": [], "recentTrips": [], "tripFeedback": [], "conversationSummary": {} }
```

This API shall aggregate all relevant memories.

7. Preference Learning Engine

The system shall automatically extract preferences from:

- Conversations
- Trip planning sessions
- Trip feedback
- Booking history

Examples:

Input: "I prefer Taj properties."

Extract:

```
{ "type": "Preference", "fact": "Likes Taj Hotels", "confidence": 0.9 }
```

Input: "Too much walking."

Extract:

```
{ "type": "Preference", "fact": "Avoid long walking itineraries", "confidence": 0.85 }
```

8. Itinerary Generation Flow

Step 1

User requests itinerary.

Step 2

Planner calls:

```
GetTravelContext(userId)
```

Step 3

Planner retrieves:

- Profile
- Preferences
- Past trips
- Feedback

Step 4

Planner performs research.

Step 5

Planner generates itinerary.

Step 6

Planner stores generated itinerary.

Step 7

User feedback is captured.

Step 8

Preference Learning Engine updates memory.

9. Future Enhancements

Phase 2

- Semantic search over past trips
- Vector memory
- Hotel preference ranking
- Restaurant preference ranking

Phase 3

- Travel Knowledge Graph
- Relationship mapping
- Cross-trip learning
- Multi-agent memory sharing

Phase 4

- Personal Travel Twin
 - Predictive trip recommendations
 - Autonomous itinerary refinement
-

10. Success Criteria

The planner should:

- Personalize every itinerary
- Reduce repeated user inputs
- Learn from every completed trip
- Improve recommendations over time
- Maintain long-term travel memory across years